CHAPTER 10

Exceptions

So far we've been developing code in Pleasantville, a wonderful place where nothing ever, ever goes wrong. Every library call succeeds, users never enter incorrect data, and resources are plentiful and cheap. Well, that's about to change.

In the real world, errors happen. Good programs (and programmers) anticipate them and arrange to handle them gracefully. This isn't always as easy as it may sound. Often the code that detects an error does not have the context to know what to do about it. For example, attempting to open a file that doesn't exist is acceptable in some circumstances and is a fatal error at other times. What's your file-handling module to do?

One approach is to use return codes to signal errors (for example, the Go language uses this pattern). In this approach, Ruby's file `open` method could return some specific value to say it failed. This value is then propagated back through the layers of calling routines until someone wants to take responsibility for it.

The problem with this approach is that managing all these error codes can be a pain. If a function calls `open`, then `read`, and finally `close` and if each can return an error indication, how can the function distinguish these error codes in the value it returns to its caller?

Ruby uses *exceptions* to help solve the problem of responding to errors. Exceptions let you package information about an error into an object. That exception object is then propagated back up the calling stack automatically until the runtime system finds code that explicitly declares that it knows how to handle that type of exception.

The Exception Class

Information about an exception is encapsulated in an object of class `Exception` or one of class `Exception`'s children. Ruby predefines a tidy hierarchy of exceptions, see https://docs.ruby-lang.org/en/master/Exception.html for the full list. As we'll see later, this hierarchy makes handling exceptions considerably easier.

The most important subclass of `Exception` is `StandardError`. The `StandardError` exception and its subclasses represent the exceptional conditions that you are going to want to capture in your code. Other subclasses of `Exception` are raised by Ruby internals or system-level problems. Almost all of the time, if you want to capture exceptions, you capture `StandardError` or one its children.